Ghost Data Still Exists

Deletion-Audit Agent: End-to-End System Explanation

HackerRank Orchestrate, October 2026

Item	Value
Repository branch	`claude/magical-carson-x2agxw`
Language	Python 3.13, standard library only (PDF backends optional)
Code size	28 product modules, 4,312 lines; 27 test files, 1,021 tests
Submitted output	`output.csv`: 518 rows: 357 INCOMPLETE, 151 COMPLETE, 10 NOT_APPLICABLE
Sample accuracy	Status 90.62%; step precision 100%, recall 85.49%, F1 92.18%
Built with	Claude Code: 2 sessions, 16 sub-agents, 38 owner-approved design decisions

1. Executive summary

Northstar Shop (fictional) receives requests to delete customer data. Its deletion workflow reports **Complete** even when personal data still sits in replicas, search indexes, analytics, support systems, exports, logs or backups, or when an active sync or a backup restore can bring it back. The task is to audit 518 such requests: decide whether each one really asked for deletion, find every copy of the person's data that existed at the audit time, apply the deletion policy, and output **COMPLETE** (with evidence-backed checks), **INCOMPLETE** (with findings, a safe ordered remediation plan and the checks that must pass afterwards) or **NOT_APPLICABLE**.

What we built. A deterministic, evidence-first audit engine with a small AI layer that runs only where rules cannot:

- A **tool layer** (12 tools): loading, identity resolution, parsers for every evidence format, a time-aware evidence index, activity-log reading, a decision-matrix planner, output validation and a scorer.
- A **case engine** that turns one request into one valid output row, using only evidence available at `audit_at`, with every step citing a participant-visible evidence ID.
- An **AI layer** on free OpenRouter models: a request-intent classifier and a screenshot reader. Every answer is cached on disk, so runs are reproducible and need no API calls once the cache exists.
- **1,021 unit tests** written by independent test agents, plus a fingerprint check that proves refactors change no behaviour.

Why this shape. The output contract is rigid (fixed operations, strict sort order, citation rules), and our measurements showed that the sample answers follow mechanical patterns. Code is the right owner for those parts. Language understanding (intent in 6 languages) and reading screenshots genuinely need a model, so the AI is confined there and cached.

Headline results (32 public samples, same settings as the submission)

Metric	Value	Meaning
Case status accuracy	90.62% (29/32)	The 3 misses are hand-made samples whose only record exists nowhere in the data
Step precision	100% (165/165)	Every step the engine emits matches the sample answer
Step recall	85.49% (165/193)	Exactly the ceiling reachable through identifier-linked records
Step F1	92.18%	
Field accuracy / exact sequence	72.88% / 34.38%	Lowered only by the missing steps shifting positions

2. The problem and the data

2.1 Inputs and output contract

File	Role
`deletion_requests.csv`	518 cases to judge: case_id, email, requested_at, reported_status (a claim), reported_at, audit_at, request_text
`sample_deletion_requests.csv` + `sample_output.csv`	32 practice cases with the correct answers
`user_identities.csv` / `user_audit_log.csv`	850 people today; 306 historical changes to email, email hash and phone
`system_activity.csv`	4,011 events (deletes, syncs, restores, checks): claims that rarely name a person
`deletion_rules.csv`	15 rules: 7 DELETE, 3 REDACT, 2 ANONYMIZE, 3 RETAIN
`evidence_files.csv` + `evidence/`	94 files: 24 SQL, 28 JSON, 1 TXT, 26 logs, 4 PDFs, 11 screenshots

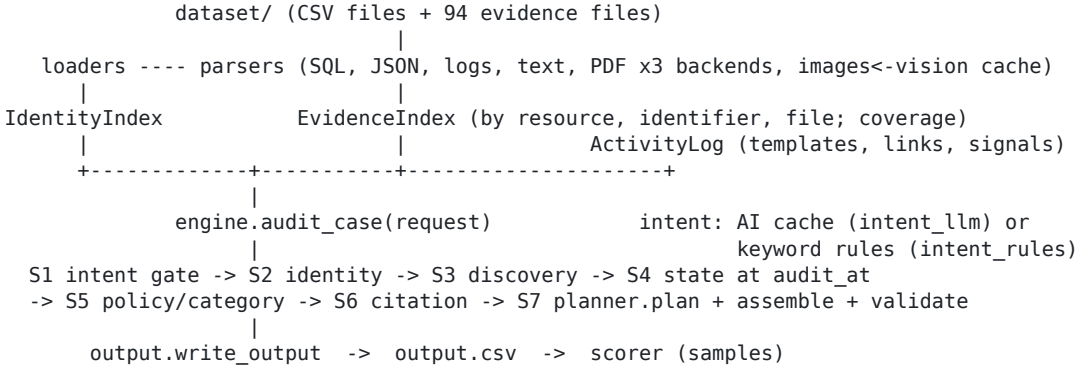
Each output row is `case_id,deletion_status,steps_json`. Steps use 15 allowed operations (4 FINDING, 5 REMEDIATION, 6 VERIFICATION), are numbered from 1, ordered by phase, then by operation order, then `system_id` and `resource_id`. Findings and COMPLETE-case checks cite evidence; remediation and post-remediation checks of INCOMPLETE cases use an empty evidence ID.

2.2 What the data analysis revealed (all measured)

Finding	Evidence	Design consequence
Claims are unreliable	11 of 17 sample 'COMPLETE' claims are really incomplete; 3 'NOT_STARTED' are complete	Claims never decide a status
Old identities matter	100 of 518 requests use an old email; one record is linked only by the hash of a previous email	Full identity history plus computed hashes
Decoys and shared names	Lookalike emails (g.silva76.jr@...), Jonas Berg / Berglund, 3 names held by 2 people	Exact matching on normalised tokens; names never link alone
Two clocks	Log files span days; 175 cases have evidence about the person dated after audit_at	Record time decides state; file capture time decides citability
Ghost copies	13 account rows exist only in a replica, never the other way round	Any store still holding a record means 'present'
Absence must be proven	Empty snapshots and full listings are the only proof of absence	A 'coverage' concept and an absence lookup
Screenshots carry facts	12 evaluation cases hinge on one screenshot or PDF, linked via catalog rows 7 min 1 s after capture	Vision cache plus link-row handling
Mechanical answers	All 32 gold rows follow one sort order; every record follows one of two step patterns	A deterministic planner and serialiser

3. Architecture

The system is a pipeline of small, separately tested modules. Everything is loaded and parsed once, then each case is decided independently from its own `audit_at`.



3.1 Directory layout

Module	Lines	Purpose (from its own docstring)
ghostaudit/__init__.py	1	Deterministic tools for the Ghost Data Still Exists deletion-audit agent.
ghostaudit/activity.py	169	T8: activity stream tools. Details and `reported_result` are claims, so nothing here decides deletion status.
ghostaudit/ai_cli.py	144	Fill or inspect the AI caches: `python -m ghostaudit.ai_cli {status,intent,vision}` (run from `code/`).
ghostaudit/engine.py	521	Deterministic case engine (engine_plan.md §2): one request in, one evidence-backed output row out.
ghostaudit/evidence_index.py	206	T7: evidence index and time filter over parsed observations.
ghostaudit/identity.py	72	Identity resolver: current snapshot plus audit-log history, with a reverse token index.
ghostaudit/intent_llm.py	205	A1: does a request text authorize deletion? Batched calls to a free OpenRouter model, cached per text.
ghostaudit/intent_rules.py	216	Keyword baseline for request intent: does the text authorize deletion?
ghostaudit/llm.py	332	OpenRouter chat client (stdlib only): retries, model fallback, defensive JSON parsing, per-attempt usage log.
ghostaudit/loaders.py	73	T1: load the participant-facing CSV files into the shared dataclasses.
ghostaudit/models.py	140	Shared data model for the deletion-audit tools. See code/docs/tools_spec.md §2.
ghostaudit/normalize.py	115	Identifier normalisation, extraction and resource-id helpers.
ghostaudit/output.py	122	T11: write, read and validate output.csv.
ghostaudit/parsers/__init__.py	185	T6 evidence parsers: turn every manifest file into Observation and Coverage objects.
ghostaudit/parsers/images.py	50	Screenshots: read facts from the vision cache `code/cache/vision/<file_id>.json`, if filled.
ghostaudit/parsers/json_records.py	101	JSON evidence: flat `records[]` lists, snapshot bundles, service resource exports, collection dumps.
ghostaudit/parsers/logs.py	269	Service logs in every line style of the dataset, mapped to resource states with EVENT_STATE_RULES.
ghostaudit/parsers/pdf_text.py	158	PDF business documents: text via pdftotext, pdftotext+BeautifulSoup or pypdf, then shared facts.
ghostaudit/parsers/sql_snapshot.py	117	SQLite snapshot exports: generic `customer_data_<store>`, typed `<collection>_<store>`, inventory.
ghostaudit/parsers/text_records.py	41	Plain-text handover records: `- ts | system | resource | record_type | result | subject`.
ghostaudit/paths.py	20	Repository paths, resolved from this file so any checkout location works.
ghostaudit/planner.py	79	T10: per-resource decision matrix and case assembly.
ghostaudit/rules.py	68	Deletion policy table and evidence collection -> data category mapping.
ghostaudit/scorer.py	265	T12: local evaluator comparing predicted rows with gold rows.
ghostaudit/steps.py	157	T9: step builder, sorter, validator and JSON serialiser.
ghostaudit/timeutil.py	60	Timestamp parsing and comparison. Every datetime returned is timezone-aware UTC.

Module	Lines	Purpose (from its own docstring)
`ghostaudit/vision.py`	341	A2: read screenshots with a free OpenRouter vision model, then map the raw facts to observation states.
`main.py`	85	Ghost Data Still Exists: audit every deletion request and write output.csv.

4. Module-by-module reference

For each module: what it does, why it was designed that way, and its public functions and classes. The function lists are generated from the source code itself; the one-line descriptions are the code's own docstrings.

4.1 Shared data model

`ghostaudit/models.py` (140 lines)

What it does. Defines the dataclasses every tool exchanges: one per CSV row type, plus **Observation** (one fact about one resource from one evidence item), **Coverage** (a file lists a full collection, so a missing record is proven absent), **SubjectProfile**, **IntentResult** and **Step**.

Why. Written first, by the main session, as the contract between agents working in parallel. Observation carries two timestamps on purpose: `observed_at` (when the fact was true) and `captured_at` (when the file existed). Names are stored as `name:`-prefixed weak tokens so they can never be mistaken for strong identifiers.

Function / class	Description
`class Request`	
`class UserIdentity`	
`class AuditChange`	
`class Activity`	
`class Rule`	
`class ManifestEntry`	
`class SubjectProfile`	Every identifier a user has had. Values are normalised (see normalize.py).
`.strong_identifiers(self)`	Tokens that identify exactly one user: user_id, emails, phones, hashes.
`class Observation`	One fact about one resource, taken from one evidence item.
`class Coverage`	The evidence item lists the full contents of a collection, so absence there proves absence.
`class IntentResult`	Whether a request authorizes deletion, and where that answer came from.
`class Step`	
`Constants / tables`	`STATES, NAME_PREFIX`

4.2 Foundation tools (T1–T5)

`ghostaudit/paths.py` (20 lines)

What it does. Resolves the repository and dataset folders from the file's own location.

Why. Works from any checkout path on macOS, Linux or Windows (AGENTS.md cross-platform rule).

Function / class	Description
`repo_root()`	The repository root (two levels above the package directory).
`dataset_dir()`	The participant-facing `dataset/` directory.
`evidence_path(file_path: str)`	Absolute path of a manifest `file_path` (relative to `dataset/`).

ghostaudit/loaders.py (73 lines)

What it does. Reads the 7 CSV files into dataclasses; sorts activity by time.

Why. The activity file is not in time order (12 rows appended at the end), so sorting is built in. Malformed rows raise errors naming the file and line.

Function / class	Description
`load_requests(name: str='deletion_requests.csv', path: str | Path | None=None, dataset_dir: str | Path | None=None)`	Deletion requests in file order.
`load_identities(path: str | Path | None=None, dataset_dir: str | Path | None=None)`	Current identity snapshot.
`load_audit_log(path: str | Path | None=None, dataset_dir: str | Path | None=None)`	Identity field changes in file order.
`load_activity(path: str | Path | None=None, dataset_dir: str | Path | None=None)`	Global activity stream sorted by (event_time, numeric activity_id suffix).
`load_rules(path: str | Path | None=None, dataset_dir: str | Path | None=None)`	Deletion policy rules.
`load_manifest(path: str | Path | None=None, dataset_dir: str | Path | None=None)`	Evidence manifest in file order; `file_path` is relative to `dataset/`.
`Constants / tables`	T

ghostaudit/timeutil.py (60 lines)

What it does. Parses every timestamp form into timezone-aware UTC; 'at or before' comparison; PDF date parsing.

Why. Equality counts as 'before' because the gold cites a file captured exactly at audit_at.

Function / class	Description
`parse_ts(s: str | datetime)`	Parse an ISO-8601 timestamp (`...Z`, `+00:00`, `T` or space separator) to aware UTC.
`at_or_before(ts: str | datetime, cutoff: str | datetime)`	True when `ts` is at or before `cutoff` (equal counts as before).
`to_iso(dt: str | datetime)`	Format as `YYYY-MM-DDTHH:MM:SSZ` (UTC, whole seconds).
`parse_human_date(s: str)`	First `DD Mon YYYY [HH MM[ SS]] [UTC]` date in `s`, as aware UTC; None when absent.

ghostaudit/normalize.py (115 lines)

What it does. Normalises emails, phones, names, hashes; extracts identifiers from free text; maps resource prefixes to systems.

Why. Exact matching on normalised tokens defeats decoys. `text_key` is shared with the intent cache so a stray space never causes a cache miss.

Function / class	Description
`norm_email(s: str)`	Strip and lowercase.
`norm_phone(s: str)`	Keep a leading `+`, then digits only: `+1 (212) 555-0171` -> `+12125550171`.
`norm_name(s: str)`	NFC, casefold and collapse whitespace; accents are kept.
`norm_hash(s: str)`	Lowercase hex without a `sha256:` prefix.

Function / class	Description
`email_hash(email: str)`	sha256 hex digest of the normalised email.
`text_key(text: str)`	Key of a request text (the intent cache key): SHA-256 hex digest of the stripped UTF-8 text.
`name_token(name: str)`	Weak identifier token for a person's name: `name:<normalised name>`.
`extract_identifiers(text: str)`	Normalised emails, phones, sha256 hashes and `user_N` ids found in `text`. Never names.
`natural_key(s: str)`	Sort key that orders `evidence_9` before `evidence_10`.
`split_resource_id(rid: str)`	`data_flow_9522` -> (`data_flow`, 9522). Raises ValueError when there is no numeric suffix.
`system_for(resource_id: str)`	Canonical system for a resource id, or None for an unknown prefix or malformed id.
`Constants / tables`	`SYSTEM_BY_PREFIX`

ghostaudit/identity.py (72 lines)

What it does. Resolves any current or historical email to every identifier the person ever had.

Why. Verified: all 518 request emails resolve (418 current, 100 historical); strong tokens map to exactly one user.

Function / class	Description
`class IdentityIndex`	Every identifier each user has held. `names` hold `name:` tokens (see models.NAME_PREFIX).
`.resolve(self, email: str)`	Profile of the user who holds or held `email` (any case); None when unknown.
`.owner_of(self, token: str)`	User ids holding a normalised token; strong tokens give at most one, names may give several.
`.profile(self, user_id: str)`	Profile for a user id, or None.

ghostaudit/rules.py (68 lines)

What it does. Loads the policy table and maps evidence collections to policy categories.

Why. Rules are data, not code, so a new category is a table row. An explicit record_type in evidence wins over the collection mapping.

Function / class	Description
`category_for(collection: str | None=None, record_type: str | None=None)`	Category from a known `record_type` (wins) or the collection table; None when unknown.
`class PolicyTable`	deletion_rules.csv keyed by data_category.
`.rule(self, category: str | None)`	The rule for a category, or None.
`.action(self, category: str | None)`	DELETE | REDACT | ANONYMIZE | RETAIN for a category, or None when it has no rule.
`Constants / tables`	`DATA_FLOW, RULE_CATEGORIES, CATEGORY_BY_COLLECTION`

4.3 Evidence parsers (T6)

ghostaudit/parsers/__init__.py (185 lines)

What it does. Dispatches each manifest file to its parser; shared helpers for identifiers and state derivation.

Why. One bad file becomes a warning, never a crash. Output is sorted deterministically. Verified: 2,897 observations and 384 coverages from 83 files.

Function / class	Description
`class ParseWarning`	A malformed or unrecognised piece of evidence (file_id and locator in the message).
`is_null(v)`	None, empty, or the literal `null`/`none` used by text formats.
`truthy(v)`	Boolean from bool/int/str forms (`true`, `1`, ...); None when undecidable.
`identifiers_of(value)`	Strong tokens in every string value (recursively) plus name tokens of name fields.
`support_exposed(rec: dict)`	Requester fields set, or a strong identifier left in the message text.
`record_state(category: str | None, rec: dict)`	State of a listed record: explicit `result` wins, otherwise derived from its fields.
`checked_state(state: str, where: str='')`	Upper-cased state if it is one of STATES, else UNKNOWN with a warning.
`make_observation(entry: ManifestEntry, *, resource_id: str, state: str, category: str | None, identifiers, source: str, fields: dict, observed_at: str | None=None, system: str | None=None, store: str | None=None)`	Observation with manifest times; system from the resource prefix, else the declared one.
`make_coverage(entry: ManifestEntry, *, system: str, collection: str, observed_at: str | None, store: str | None=None)`	Coverage record with manifest times.
`parse_file(entry: ManifestEntry)`	Parse one manifest entry with the parser for its extension (modules check content shape).
`obs_sort_key(o: Observation)`	Deterministic sort key for observations.
`cov_sort_key(c: Coverage)`	Deterministic sort key for coverages.
`parse_all(manifest: list[ManifestEntry] | None=None)`	Parse every manifest entry; one bad file becomes a warning instead of a crash.
`Constants / tables`	`NAME_FIELDS, NON_IDENTITY_FIELDS, ANALYTICS_ID_FIELDS, TOMBSTONE_ID_FIELDS, SUPPORT_REQUESTER_FIELDS, SUPPORT_TEXT_FIELDS`

ghostaudit/parsers/sql_snapshot.py (117 lines)

What it does. Loads SQL dumps into in-memory SQLite: generic record_json tables with replicas, typed tables, inventory schema.

Why. Keeps the store (primary, read_replica, replica_eu_west, ...) on every observation, which is what makes ghost-copy detection possible.

Function / class	Description
`parse(entry: ManifestEntry, path: Path)`	One observation per row, one coverage per table (per scope system for inventories).
`Constants / tables`	`GENERIC_BASE, TYPED_BASES, COLLECTION_SYSTEM, SYSTEM_COLLECTION, SCOPE_TABLE, ROW_TIME_FIELDS`

ghostaudit/parsers/json_records.py (101 lines)

What it does. Handles flat record lists, snapshot bundles, service resource exports and collection dumps.

Why. Uses each snapshot's or item's own time; only bundle snapshots count as full listings (coverage).

Function / class	Description
`parse(entry: ManifestEntry, path: Path)`	Dispatch on the document shape; only bundle snapshots are full listings (coverage).
`Constants / tables`	`HISTORY_COLLECTION, ITEM_TIME_FIELDS, SECTION_COLLECTION`

ghostaudit/parsers/logs.py (269 lines)

What it does. Parses 7 log line styles, joins started/completed lines by trace ID, maps events to states.

Why. Lines of different traces are interleaved, so the join is essential. Every one of the 2,211 log lines is either parsed or recognised as noise.

Function / class	Description
`parse_log_line(line: str)`	Split a line into ts, level, service, trace_id, event, kv and style; None if unparseable.
`is_start_event(event: str)`	True for *_started log events (feed the trace join).
`parse(entry: ManifestEntry, path: Path)`	One observation per state-bearing line; noise and start lines only feed the join.
`Constants / tables`	`NOISE_EVENTS, EVENT_STATE_RULES`

ghostaudit/parsers/text_records.py (41 lines)

What it does. Parses the pipe-delimited shift handover file.

Why. Simple, direct mapping; uppercase subjects are normalised.

Function / class	Description
`parse(entry: ManifestEntry, path: Path)`	One observation per record line; the subject is matched as normalised identifiers.
`Constants / tables`	`FIELD_NAMES`

ghostaudit/parsers/pdf_text.py (158 lines)

What it does. Extracts PDF text with three interchangeable backends (pdftotext, BeautifulSoup over pdftotext XHTML, pypdf) and pulls out facts.

Why. Owner requested multiple backends; Selenium was rejected because a browser can show a PDF but not extract its text. All three backends give identical facts on all 4 PDFs.

Function / class	Description
`extract_lines(path: str | Path, backend: str='pdftotext')`	Non-empty text lines of a PDF in reading order, from the chosen backend.
`extract_facts(lines: list[str])`	Sorted resource IDs, user ids, emails, phones, hashes, ORD refs, other refs and ISO dates.
`document_category(lines: list[str])`	
`parse(entry: ManifestEntry, path: Path)`	FOUND fact observations: one per resource ID, or one document-level (resource_id '').
`Constants / tables`	`BACKENDS, DOC_CATEGORY_RULES, CATEGORY_SYSTEM`

ghostaudit/parsers/images.py (50 lines)

What it does. Turns cached screenshot facts into observations.

Why. Screenshots cannot be read by rules, so the vision model fills a cache once; parsing stays deterministic.

Function / class	Description
`cache_path(file_id: str)`	Location of a cached item.
`parse(entry: ManifestEntry, path: Path)`	Observations from the cached vision items; empty with one warning when no cache exists.

4.4 Evidence reasoning (T7–T8)

ghostaudit/evidence_index.py (206 lines)

What it does. Indexes observations; finds a person's records at a cutoff; latest state; per-store state; citation choice; absence proof.

Why. Encodes the two-clock rule. `citable` picks the latest file captured by audit_at (agrees with 74 of 75 gold citations). `absent_in` never lets an older listing override newer evidence.

Function / class	Description
`class SubjectMatch`	An observation linked to a subject and why: one of REASONS.
`is_resource_id(value: object)`	True for a resource-shaped string such as `search_052`; `user_N` ids are identifiers, not resources.
`class EvidenceIndex`	Observations indexed by resource id, identifier token, file id and field references.
`.build(cls, manifest: list[ManifestEntry] | None=None)`	Parse every manifest file once (via `parsers.parse_all`) and index the result.
`.for_subject(self, profile: SubjectProfile, cutoff: str, extra_links: Mapping[str, Iterable[str]] | None=None, *, strict: bool=False)`	Observations linked to the subject at `cutoff`, in history order, each with its reason.
`.history(self, resource_id: str, cutoff: str, *, strict: bool=False)`	Observations of a resource with `observed_at <= cutoff`, oldest first.
`.state_at(self, resource_id: str, cutoff: str, *, strict: bool=False)`	The latest observation of a resource at or before `cutoff`.
`.states_by_store(self, resource_id: str, cutoff: str, *, strict: bool=False)`	The latest observation per store (None keyed `"-"`), for replica drift.
`.citable(self, resource_id: str, cutoff: str)`	File with the latest manifest `captured_at <= cutoff` among files observing the resource.
`.coverage(self, system_id: str, cutoff: str)`	Full-collection listings for a system with `observed_at <= cutoff`, newest first.
`.absent_in(self, resource_id: str, system_id: str, cutoff: str)`	The newest citable coverage of `system_id`, if it proves the resource absent.
`.in_file(self, file_id: str)`	Every observation from one evidence file, in history order.
`.with_token(self, token: str, cutoff: str | None=None)`	Observations carrying an identifier token (strong or `name:`), optionally up to `cutoff`.
`.resource_ids(self)`	Every indexed resource id, sorted.
Constants / tables	REASONS

ghostaudit/activity.py (169 lines)

What it does. Parses the ~36 activity detail templates into resource IDs and signals (flow still enabled, restore possible, repopulated, non-deletion, identity link).

Why. Activity rows are claims and never decide a verdict; they supply links (12 catalog rows) and corroboration.

Function / class	Description
`class ActivityDetails`	What a details string says. `claimed_result` is the row's reported_result: a claim, not a state.
`parse_details(activity: Activity)`	Match the details text against the known templates; unknown text gives `template="unknown"`.
`class ActivityLog`	The global activity stream, time-ordered, with parsed details and a resource index.
`.details(self, activity: Activity | str)`	Parsed details for an activity or activity_id.
`.unknown(self)`	Activity ids whose details match no known template.
`.for_resources(self, ids: str | Iterable[str], until: str | None=None)`	Rows whose resource_id or details mention any of `ids`, at or before `until`, in time order.
`.window(self, start: str | None=None, end: str | None=None)`	Rows with start <= event_time <= end (either bound may be None).
`.with_signal(self, signal: str, until: str | None=None)`	Rows carrying a details signal (see SIGNALS), at or before `until`.
`.links(self, until: str | None=None)`	resource_id -> identifier tokens from LINK_RESOURCE_IDENTITY rows (emails, user ids,
`.non_deletion_flags(self, until: str | None=None)`	resource_id -> latest CLASSIFY_REQUEST activity_id marking the request as non-deletion.
`Constants / tables`	`SIGNALS`

4.5 Planning, output and scoring (T9–T12)

ghostaudit/steps.py (157 lines)

What it does. Operation table, step builder, canonical sort, validator and JSON serialiser.

Why. Reproduces all 32 gold rows exactly; the validator rejects anything outside the contract, including steps out of canonical order.

Function / class	Description
`class OpSpec`	
`make_step(operation: str, system_id: str, resource_id: str, evidence_id: str='')`	A Step with phase and state taken from OPERATIONS; step_no is 0 until numbered.
`step_key(step: Step)`	Every field except step_no: (phase, system_id, resource_id, operation, state, evidence_id).
`sort_and_number(steps: Iterable[Step])`	Drop exact duplicates, sort by (phase, operation rank, system_id, resource_id), number from 1.
`validate(status: str, steps: Sequence[Step])`	Contract violations for one output row; an empty list means valid.
`to_json(steps: Iterable[Step])`	Compact JSON array with keys in schema order and non-ASCII kept as is.
`from_json(text: str)`	Parse a steps_json array; raises ValueError on malformed JSON or step objects.
`Constants / tables`	`PHASES, STATUSES, KEYS, EVIDENCE_ID_RE`

ghostaudit/planner.py (79 lines)

What it does. The per-record decision matrix and case assembly.

Why. Derived from all 193 gold steps: an already-compliant record gives one verification; a non-compliant one gives finding, fix and verification. Undefined combinations raise instead of guessing.

Function / class	Description
`class ResourcePlan`	The steps one resource contributes to a case.
`plan(system_id: str, resource_id: str, kind: str, current_state: str, evidence_id: str='')`	Apply the T10 decision matrix to one resource; raises ValueError for undefined combinations.
`assemble(plans: Iterable[ResourcePlan])`	Merge resource plans into (status, numbered steps): INCOMPLETE if any finding, else COMPLETE.
`Constants / tables`	`KINDS`

ghostaudit/output.py (122 lines)

What it does. Writes, reads and validates output.csv; CLI checker.

Why. Guarantees exactly one valid row per request, in request-file order.

Function / class	Description
`write_output(rows: Iterable[Row], path: str | Path | None=None)`	Write (case_id, status, steps) rows in the given order; defaults to <repo>/output.csv.
`order_like_requests(rows: Iterable[Row], requests: Sequence[Request])`	Rows reordered to the request file order; unknown or duplicate case_ids raise ValueError.
`read_rows(path: str | Path)`	Strictly read an output-format CSV into {case_id: (status, steps)}.
`validate_output(path: str | Path, requests: Sequence[Request])`	Every contract violation in an output file, checked against the request list.
`main(argv: Sequence[str] | None=None)`	CLI: validate an output file and print its violations.
`Constants / tables`	`HEADER`

ghostaudit/scorer.py (265 lines)

What it does. Local evaluator: status accuracy, step precision/recall/F1, field and sequence accuracy, per-case diffs.

Why. Feeds the evaluation report; mutation tests prove each metric reacts only to what it measures.

Function / class	Description
`class PRF`	Micro precision/recall/F1 counts over step multisets.
`.precision(self)`	
`.recall(self)`	
`.f1(self)`	
`class CaseDiff`	How one predicted case differs from gold.
`class Report`	Every metric of one scoring run; see to_markdown().
`.status_accuracy(self)`	
`.field_accuracy(self)`	
`.field_overall(self)`	
`.sequence_accuracy(self)`	

Function / class	Description
`.to_markdown(self, max_diffs: int | None=None)`	Summary tables plus the per-case diff list.
`score(pred_rows: Cases | str | Path, gold_rows: Cases | str | Path)`	Score predictions over the gold case set; a missing prediction is wrong everywhere.
`main(argv: Sequence[str] | None=None)`	CLI: score a prediction CSV against a gold CSV.
`Constants / tables`	`FIELDS, MISSING`

4.6 Case engine and entry point

ghostaudit/engine.py (521 lines)

What it does. Decides one request: intent gate, identity, discovery, state, policy, citation, planning, validation. Every decision leaves a note.

Why. All behaviour beyond the plan was approved by the owner (decisions 5–31). Data tables (category hints, state rules, retention fields) keep rules editable without code changes.

Function / class	Description
`class CategoryHint`	Turn a `base` category into `category` when one `when` clause matches a single observation.
`class CaseResult`	One output row plus the notes explaining how it was reached.
`.row(self)`	(case_id, status, steps) for output.write_output.
`note_kind(note: str)`	The kind prefix of a note ("link_fallback: ..." -> "link_fallback").
`llm_cache_lookup()`	`intent_llm.cached` (cache-only, never calls the API), or None when that module is absent.
`class Context`	Everything a run loads once: identities, evidence, activity, policy and the intent source.
`.build(cls, intent_mode: str='auto')`	Load every participant-facing file. `cache` mode fails here when intent_llm is absent.
`.intent(self, text: str)`	Intent from the LLM cache (auto, cache) or the keyword baseline (auto fallback, rules).
`subject_links(request: Request, profile: SubjectProfile, log: ActivityLog)`	LINK_RESOURCE_IDENTITY rows naming the subject, logged in [requested_at, audit_at].
`identifier_free_resources(request: Request, profile: SubjectProfile, ctx: Context)`	Hook for records with no identifier left (plan §2.4, T2/T3). Out of scope for now: no resources.
`audit_case(request: Request, ctx: Context)`	Decide one request: NOT_APPLICABLE, or the evidence-backed COMPLETE/INCOMPLETE plan.
`run(requests: Sequence[Request], ctx: Context)`	Audit every request in order.
`Constants / tables`	`INTENT_MODES, CATEGORY_BY_PREFIX, CATEGORY_BY_LINK_TEMPLATE, CATEGORY_HINTS, RETENTION_FIELDS, STATE_RULES, COMPLIANT_STATE, PERMITTED_IDENTIFIERS, NAME_EXPOSED_CATEGORIES, PRESENT_STATE, SAFEST_ROW, UNSUCCESSFUL_RESULTS`

ghostaudit/intent_rules.py (216 lines)

What it does. Keyword intent baseline in 6 languages.

Why. Deterministic fallback when no AI cache exists. Treats 'verify my deletion' as deletion (as the gold does); negation wins. Flags exactly the 10 non-deletion requests.

Function / class	Description
`detect_language(text: str)`	ISO 639-1 code from script (ko, ja) or Latin keyword cues (pt, es, fr), else en.
`classify(text: str)`	Keyword verdict on whether `text` authorizes deletion (source="rules").
`Constants / tables`	`SOURCE, NEGATION, KEEP_WISH, DELETION, ACCOUNT_DELETION, VERIFICATION, IMPERATIVE, TOPICS`

`main.py` (85 lines)

What it does. CLI: audits every request, writes output.csv, validates it and prints a summary.

Why. Intent modes: auto (AI cache, else rules), rules, cache. A missing input prints a one-line error.

Function / class	Description
`main(argv: list[str] | None=None)`	Run the engine over a request file, write and validate the output CSV.

4.7 AI layer (free OpenRouter models)

`ghostaudit/llm.py` (332 lines)

What it does. OpenRouter client: retries with growing waits, fallback model, defensive JSON parsing, usage logging.

Why. Standard library only (no SDK). Never logs the key or prompt text. 4 tries per model, then the task's fallback model gets 4 more.

Function / class	Description
`class LLMError`	No valid model output. `fatal`: later calls will fail too; `rate_limited`: retry budget spent on 429s;
`class UsageRecord`	Totals for one chat_json call (every attempt that got a model response).
`cache_root()`	`code/cache/`; created lazily by the first write, never by a read.
`usage_log_path()`	Location of the usage log.
`write_json(path: Path, obj)`	Stable JSON (sorted keys, UTF-8, indent 1) written atomically.
`image_part(path: str | Path)`	OpenAI-style image content part with a base64 data URL.
`parse_json_reply(content)`	First JSON object in a reply: think blocks and code fences stripped, surrounding prose ignored.
`class OpenRouterClient`	Chat-completions client for OpenRouter's OpenAI-compatible endpoint.
`.has_key(self)`	Whether OPENROUTER_API_KEY is set (never reveals it).
`.reachable(self, timeout: float=5.0)`	Whether the base URL answers at all (any HTTP status counts); no key is sent.
`.available(self)`	(True, detail) when a key is set and the base URL is reachable.
`.chat_json(self, messages: list[dict], *, model: str | None=None, task: str, schema_check: Callable[[dict], None] | None=None, kind: str='text')`	One JSON reply at temperature 0 from `model` (default: the main model of `kind`, text or vision).
`Constants / tables`	`DEFAULT_BASE_URL, DEFAULT_TEXT_MODEL, DEFAULT_VISION_MODEL, DEFAULT_TEXT_FALLBACK_MODEL, DEFAULT_VISION_FALLBACK_MODEL, TASK_KINDS, ATTEMPTS_PER_MODEL, APP_TITLE, AUTH_STATUS, FATAL_STATUS, IMAGE_MIME`

ghostaudit/intent_llm.py (205 lines)

What it does. Batched intent classification with a versioned prompt and synthetic few-shot examples; one cache file per text.

Why. Cache keyed by text hash and prompt version: reruns cost nothing and give identical answers.

Function / class	Description
`cache_key(text: str)`	SHA-256 hex digest of the stripped UTF-8 text (shared with engine.text_key, owner decision 26).
`cache_path(key: str)`	Location of a cached item.
`cached(text: str)`	The cached result for `text` under the current PROMPT_VERSION, or None.
`build_messages(texts: list[str])`	System definition, one synthetic few-shot exchange, then the batch.
`validate(data: dict, n: int)`	Normalised results by id (1..n); raises ValueError naming the first problem.
`classify_all(texts: Iterable[str], *, client: llm.OpenRouterClient | None=None, batch_size: int=10, force: bool=False, report: dict | None=None)`	Results keyed by cache_key(text). Misses are sent in batches sorted by key and cached per text.
`Constants / tables`	`PROMPT_VERSION, REQUEST_TYPES, DELETION_TYPES, MAX_QUOTE, SYSTEM_PROMPT, FEW_SHOT`

ghostaudit/vision.py (341 lines)

What it does. Reads each screenshot once, saves raw labels, maps labels to states with fixed rules, cross-checks catalog link rows.

Why. The model only transcribes; deterministic rules decide states, so the reasoning stays auditable (owner decisions 7, 17–19, 32).

Function / class	Description
`raw_path(file_id: str)`	
`screenshots(manifest: list[ManifestEntry] | None=None)`	Image entries of the manifest, in natural file_id order.
`messages_for(path: Path)`	
`check_extraction(data: dict)`	schema_check for the model reply: a records list whose entries each carry a resource_id.
`normalize_extraction(data: dict)`	Coerce the model reply to the raw schema (strings and string lists only).
`links_for(entry: ManifestEntry, activity: list[Activity])`	Data-catalog link rows timestamped shortly after this capture.
`map_state(category: str | None, rec: dict, strong: set[str])`	(state, rule applied) for one merged raw record, from the deterministic tables.
`map_extraction(entry: ManifestEntry, extraction: dict, links: list[dict])`	Observation items for parsers/images.py, plus a report (aliases, dropped records, cross-check).
`cross_check(items: list[dict], links: list[dict])`	Per link row: is its resource extracted, and do the identifiers shown agree with the link?
`pending(manifest: list[ManifestEntry] | None=None, force: bool=False)`	Screenshots that need a model call (no current raw extraction cached).
`fill_cache(manifest: list[ManifestEntry] | None=None, *, client: llm.OpenRouterClient | None=None, force: bool=False, activity: list[Activity] | None=None)`	One call per screenshot without a cached raw extraction (all of them with `force`); cached raw

Function / class	Description
`Constants / tables`	`PROMPT_VERSION, IMAGE_EXTS, RESOURCE_ID_RE, LINK_EVENT, LINK_WINDOW, LIST_KEYS, CATEGORY_BY_PREFIX, LABEL_STATES, NEGATED_LABEL_STATES, RISKY_STATES, LABEL_FIELD_WORDS, RESTORE_BLOCK_WORDS, RESTORE_ALLOW_WORDS, RESTORE_REFUSAL_WORDS, NEGATIONS, EXPOSURE_CATEGORIES, SYSTEM_PROMPT, USER_PROMPT`

ghostaudit/ai_cli.py (144 lines)

What it does. `status` / `intent` / `vision` commands to inspect and fill the caches.

Why. Exits cleanly (code 2) without a key or network, a rejected key or no credit.

Function / class	Description
`cmd_status(args, client: llm.OpenRouterClient)`	Print key presence (never the value), reachability, models and cache coverage.
`cmd_intent(args, client: llm.OpenRouterClient)`	Fill the intent cache for the request files.
`cmd_vision(args, client: llm.OpenRouterClient)`	Fill the screenshot cache.
`main(argv: list[str] | None=None)`	Command-line entry point.
`Constants / tables`	`DEFAULT_REQUESTS, NOT_READY`

5. How a case is decided

5.1 The per-case pipeline

Stage	What happens	Key rule
S1 Intent	AI cache answer if present, else keyword rules	Non-deletion → NOT_APPLICABLE with []. 'Verify my deletion' counts as deletion
S2 Identity	Resolve the request email	Current or historical email; any case
S3 Discovery	Records linked by email, phone, hash or user_id, plus one hop via shared resource IDs; catalog link rows in the case window	Names never link alone; records settled by an earlier request are skipped (decision 5)
S4 State	Latest observation with observed_at ≤ audit_at	A replica copy in the same file means present (ghost copy)
S5 Policy	Category from evidence, prefix table, then field hints → planner kind	email_opt_in → marketing profile (DELETE); attachment → SUPPORT_ATTACHMENT; legacy contact copy → SECONDARY_IDENTIFIER
S6 Citation	Latest file captured ≤ audit_at that shows the decided state; absence via a full listing	Never cite a file created after the audit; a late sighting cites the failed deletion row (decision 6)
S7 Plan	Decision matrix per record, assemble, validate	Any finding → INCOMPLETE; otherwise COMPLETE

5.2 Policy → planner kind

Policy categories	Kind	Note
USER_PROFILE, SECONDARY_IDENTIFIER, SEARCH_INDEX_ENTRY, CACHE_ENTRY, SUPPORT_ATTACHMENT, USER_EXPORT, MARKETING_PROFILE	DELETE	
SUPPORT_MESSAGE, EMAIL_DELIVERY_LOG, ACCOUNT_TOMBSTONE	REDACT	Tombstone keeps only user_id; a visible name counts as exposed
ANALYTICS_PROFILE, PRODUCT_TELEMETRY	ANONYMIZE	
SECURITY_AUDIT_LOG, LEGAL_HOLD_RECORD	RETAIN	Only while retention or hold dates are after audit_at
BACKUP_ARCHIVE	BACKUP_ARCHIVE	Restore allowed → report and block
DATA_FLOW (no rule; a recreation path)	DATA_FLOW	Enabled → report and disable

5.3 The decision matrix (derived from all 193 gold steps)

Kind	State at audit	Steps
DELETE	FOUND / IDENTIFIABLE	REPORT_REMAINING_RECORD → DELETE_RECORD → VERIFY_RECORD_ABSENT
DELETE	NOT_FOUND	VERIFY_RECORD_ABSENT
REDACT	IDENTIFIABLE / REDACTED	REPORT_EXPOSED_PII → REDACT_RECORD → VERIFY_RECORD_REDACTED / verification only
ANONYMIZE	IDENTIFIABLE / ANONYMIZED	REPORT_EXPOSED_PII → ANONYMIZE_RECORD → VERIFY_RECORD_ANONYMIZED / verification only
RETAIN	FOUND	VERIFY_RECORD_PRESENT
DATA_FLOW	ACTIVE / DISABLED	REPORT_ACTIVE_DATA_FLOW → DISABLE_DATA_FLOW → VERIFY_DATA_FLOW_DISABLED / verification only
BACKUP_ARCHIVE	ACTIVE / BLOCKED	REPORT_RESTORE_RISK → BLOCK_RESTORE → VERIFY_RESTORE_BLOCKED / verification only

Remediation follows the operation table order, so data flows are disabled and restores blocked before records are changed, which prevents deleted data from being recreated mid-fix.

6. The AI layer

Part	Job	Main model	Fallback
A1 Intent	Does the request authorize deletion? (6 languages)	google/gemma-4-31b-it:free	nvidia/nemotron-3-super-120b-a12b:free (session 2 ran with nemotron-3-ultra-550b-a55b:free via environment variable, decision 38)
A2 Vision	Read the facts shown in 11 admin screenshots	nvidia/nemotron-3.5-super-vl:free	google/gemma-4-31b-it:free

- **Why free OpenRouter models:** the owner chose free models instead of paid APIs. The OpenRouter key was kept in environment settings and is never printed, logged or committed.
- **Retry and fallback (decisions 16, 29–31):** each wait is at least 2, 4, 8 seconds; 4 tries per model on rate limits, outages or an unavailable model; then the fallback gets 4 tries; then the item is left for the next run, never guessed.
- **Determinism:** temperature 0, stable batching, and every answer cached per text hash and prompt version, so reruns make no calls and give identical results.
- **Prompts:** the intent prompt defines deletion (including verification requests), lists non-deletion types, says negation wins, and uses 8 synthetic few-shot examples (never copied from the dataset). The vision prompt asks only for a transcription; fixed rules map labels to states.
- **Usage logging:** one line per attempt with requested and actual model and token counts; the usage report is built from it.

6.1 What actually ran

The key only reaches new sessions, so a second Claude Code session ran the AI steps. The intent run cached 190 distinct texts (271 of 550 request rows) before the owner stopped it for the deadline; it agreed with the keyword baseline on all 271 rows. Free-tier rate limits caused 103 HTTP 429 attempts on Gemma and 16 HTTP 404 attempts on the configured fallback, so the owner switched the fallback to Nemotron Ultra. The screenshot reader was not run. In total 148 attempts used 83,348 tokens at $0.00. Because the AI answers matched the rules, `output.csv` is byte-identical with either intent source.

7. Key design choices and why

Choice	Why
Deterministic engine, AI only for intent and screenshots	The output contract and gold patterns are mechanical; language and images are not. Cheaper, reproducible, explainable
Python standard library only	Runs anywhere with no installs; PDF backends are optional extras
Evidence first; claims are features	Workflow statuses and activity results proved unreliable (11 of 17 'COMPLETE' claims false)
Two clocks per observation	State must reflect audit_at; citations must use files that existed then
Exact identifier matching; names are weak	Lookalikes and shared names exist in the data
Coverage objects for absence	A deletion can only be verified by a listing that should contain the record
Add identifier-free records only when unique	Timing-based linking recovered only about 5 of 20 sample accounts; a miss costs one verification step, never a status
Every design decision approved by the owner	38 numbered decisions recorded in engine_plan.md, each backed by a data check
Caching all AI answers	Free models are rate limited and the router can be random; cached answers make runs reproducible

7.1 Owner decisions (summary)

#	Decision
1–4	Free OpenRouter models; build in session 1 and run the AI in a new session; chosen models; weak links only if unique
5–6	Skip records settled by an earlier request; a late-exported sighting before the audit cites the failed deletion row
7–10	Unlabelled backup = restore risk; no-evidence fallback = NOT_APPLICABLE; link rows only inside the case window; interim 'assume present' for screenshot-only records
11–15	Intent prompt approved; blank-ID index bug fixed; scope rules for decisions 5 and 6
16–31	Retry safety; screenshot label rules; five engine and intent bug fixes; late-absence handling; CLI and cache fixes; fallback models, attempt budget and triggers
32–37	Finer retry and label details (recorded; not implemented because the code was frozen for submission)
38	Session 2: intent fallback switched to nemotron-3-ultra-550b via environment variable

8. Exception cases and mitigations

Exception	Mitigation
Wrong workflow claims	Never used as verdicts; only as a weak linking feature
Old emails and hashes	Identity history plus hashes of every past email
Decoys, lookalikes, shared names	Exact normalised tokens; names only corroborate
Evidence dated after the audit	observed_at for state, captured_at for citations; metamorphic test proves later evidence never changes an answer
Ghost copies in replicas	Per-store state in the same file
Proving absence	Newest pre-audit full listing that does not mention the record; never overrides newer evidence
Records with no identifiers left	Status decided without them; added only when unambiguous
Unreachable gold records (3)	Accepted as the ceiling; no invented IDs
Category edge cases	Field-hint table (opt-in → marketing, attachment, legacy contact copy)
Screenshots and conflicting screens	Vision cache plus catalog link rows; conflicts flagged (case_63)
Multilingual and tricky intent	Explicit definition, negation handling, keyword cross-check
Data coming back (syncs, restores)	Disable and block before deleting; repopulation signals
Same person, several cases	Each case recomputed from its own audit_at
Unrepresentative samples	Independent tests, custom cases, no-hardcoding guard

9. Testing and quality assurance

- **Independent testers:** the test agents never tested code they wrote. Five test agents produced 1,021 tests across 27 files (about 15 s).
- **Bugs found by tests and fixed:** pipe-log levels; absence proof overriding newer evidence; blank-ID linking; citations showing the wrong state; an endless retry on 'retry after 0'; keyword false refusals; an ambiguous-email crash; tombstone names; and more.
- **Fingerprint check:** a hash of all parser output, per-case links and citations, activity parsing and PDF facts, compared before and after the cleanup refactor, proved behaviour unchanged.
- **No-hardcoding guard:** a test fails if product code contains literal case, evidence or activity IDs.
- **Metamorphic test:** adding later-dated evidence must never change an answer.
- **AI tests:** a local fake server and temporary caches; no real API calls and no real waiting.

9.1 Custom evaluation cases

Case	Result
Non-deletion requests in 6 languages, negation, verify-my-deletion	Pass
Evidence after audit_at added to a case	Output unchanged (pass)
Account present only in a replica	INCOMPLETE with REPORT_REMAINING_RECORD (pass)
Pre-audit sighting in a late-exported file	Finding cites the failed deletion row (pass; real case_126)
Record from an earlier request	Skipped (pass; sample case_17 now exact)
Email held by two users	Safe fallback row instead of a crash (pass)

10. Running it, limitations, and how it was built

10.1 Commands (from the repository root)

```
python3 code/main.py                      # output.csv for all 518 cases (AI cache if present, else rules)
python3 code/main.py --intent rules       # deterministic keyword intent (used for the submission)
python3 code/main.py --requests sample_deletion_requests.csv --output /tmp/sample.csv
cd code && python3 -m ghostaudit.scorer --pred /tmp/sample.csv --gold ../dataset/sample_output.csv
cd code && python3 -m ghostaudit.ai_cli status|intent|vision      # needs OPENROUTER_API_KEY
python3 -m unittest discover -s code/tests -t code                # 1,021 tests
```

10.2 Limitations and next steps

- Records with no identifier left (25 of 111 sample records) are mostly not recovered; a better linker could raise recall.
- The screenshot reader was never run; 9 screenshot-only cases use the approved 'assume present, cite the link row' stand-in.
- The AI intent run covered 271 of 550 rows; finishing it would only confirm intent (all answers so far match the rules).
- Decisions 32–37 are recorded but not implemented.
- Three sample answers reference records that exist nowhere in the participant data.

10.3 Development process

The system was built with Claude Code in two sessions. The main session analysed the data, wrote specifications, and coordinated 16 sub-agents: 4 coders for the tools, coders for the engine and AI client, 5 independent test writers, a cleanup agent and fix agents. Every agent report was verified by re-running its checks before committing. After the owner's instruction that every decision must go through them, each open question was put to the owner with a recommended option and the supporting data. The second session, which could see the API key, ran the AI steps. Every turn is logged in log.txt, and both sessions' transcripts are in agent-trace.zip.